



OBJECT ORIENTED SOFTWARE ENGINEERING

Ms. Archana A

Department of Computer Applications

OBJECT ORIENTED SOFTWARE ENGINEERING

Design Concepts...

Archana A

Department of Computer Applications

Design concepts provide the **necessary framework**
“to get the thing on right way”.

1. Abstraction.
2. Architecture.
3. Patterns.
4. Separation of concerns.
5. Modularity
6. Information Hiding.
7. Functional independence.
8. Refinement
9. Aspects
- 10. Refactoring.**
- 11. Object Oriented design concepts**
- 12. Design Classes.**

- Refactoring is a **re-organization technique** that simplifies the design (or code) of a component **without changing its function or behavior**.
- Refactoring improves its internal structure

- When software is refactored, the **existing** design is **examined for** :
 - redundancy
 - unused design elements
 - **inefficient** or unnecessary algorithms
 - **poorly constructed** or inappropriate data structures
 - any other design failure that can be corrected to yield a **better design**

- For Example:

Scenario 1: Feature Addition Difficulty

- **Problem:** When adding a new feature, the developer encounters significant challenges due to the existing code structure. Modifying the current codebase might lead to unintended side effects or introduce complexity.
- **Refactoring Example:** The developer might refactor the codebase to introduce an abstraction layer or design pattern that better separates concerns. This allows for easier integration of the new feature without impacting existing functionality.

- Another Example:

Scenario 2: Code Review Identifies Issues

Problem: During a code review, a team member identifies **potential design flaws** like **code duplication, redundant classes, or violation of specific design principles**.

Refactoring Example: The team might decide to refactor the code to remove duplication, merge redundant classes, or apply appropriate design patterns to improve the overall code structure and maintainability.

- The object-oriented (OO) paradigm is **widely used** in modern Software Engineering.
- Very common aspects of object orientation are :
 - **inheritance,**
 - **Polymorphism,**
 - **Design classes and its objects.**

1. **Inheritance** → all responsibilities of a superclass is immediately inherited by all subclasses
2. **Polymorphism** → we can define polymorphism as **the ability of a message to be displayed in more than one form.**
 - A real-life example of polymorphism is a person who at the same time can have different characteristics. Like a man at the same time is a father, a husband and an employee

3. Design classes → **Entity classes**, **Boundary classes**, **Controller classes**

- **Entity Classes**: Analysis classes are **refined during design** to become entity classes.
- **Boundary classes** are developed during design to create the **interface** (for eg; interactive screen, printed report) that the user view and interacts with the software.

- **Controller classes** are designed to manage:
 - Creation or update of entity objects
 - The instantiation of boundary objects as they obtain information from entity objects.
 - Complex communication between sets of objects
 - Validation of data communicated between **objects**, between **users** and the **application**.

- The **requirements model** defines a set of **analysis classes**. Each describes some element of the **problem domain, focusing on aspects** of the problem that are user visible.
- As the **design model evolve**, we will define a set of ***design classes, that refine the analysis classes*** by providing design detail that will enable the classes to be implemented

Each design class is reviewed to ensure that it is “**well-formed.**”

There are Four characteristics of a well-formed design class:

1. **Complete and sufficient** – a design class should be the complete encapsulation of all attributes and methods to exist for the class

2. **Primitiveness** – methods associated with a design class should be **focused on accomplishing one service for the class**. Once service has been implemented with a method, the class **should not provide another way to accomplish** the same thing.

3. **High cohesion** – a cohesive design class has a small, focused set of responsibilities and single-mindedly applies attributes and methods to implement those responsibilities.

4. **Low coupling** – collaboration should be kept to an acceptable minimum. If a design model is **highly coupled** (all design classes collaborate with all other design classes), the **system is difficult to implement, to test, and to maintain over time**.



THANK YOU

Ms. Archana A

Department of Computer Applications

archana@pes.edu

+91 80 6666 3333 Extn 392